Aegis Security Evaluation

Detailed Bilingual Speaking Script for Each PPT Slide

Aegis 项目汇报逐页中英双语详细口播稿

Purpose 用途 This document is designed for live presentation rehearsal. Each slide section includes a speaking goal, a detailed Chinese script, a detailed English script, and plain-language explanations of the key points so that you can present fluently and answer follow-up questions with confidence.

整体使用方式 / How to Use This Script

- **Step 1：**先背熟 Slide 1、5、10、13，这四页决定你的主线、数据、根因和收尾。
  - *English note:* Memorize Slides 1, 5, 10, and 13 first. They anchor your framing, numbers, root causes, and closing.
- **Step 2：**其余页面不需要逐字背，但必须知道每页的目标是什么、为什么放在这里。
  - *English note:* The other slides do not need word-for-word memorization, but you should know the purpose of each page.
- **Step 3：**现场如果时间紧，可以优先讲 Slide 1、3、5、6、9、10、13。
  - *English note:* If time is short, prioritize Slides 1, 3, 5, 6, 9, 10, and 13.
- **Current data convention：**本稿采用仓库最新口径：Track A = 84 / 65 / 18 / 3；Track B = 80 / 73 / 5 / 0。
  - *English note:* This script follows the latest repository convention: Track A = 84 / 65 / 18 / 3; Track B = 80 / 73 / 5 / 0.

一句总主线 / One-sentence through-line

中文：你整场都可以围绕这句话展开：Aegis 很快，也确实能拦住一部分显式攻击，但它还不是一个足够强的语义安全边界。

English: You can organize the whole talk around one sentence: Aegis is fast and genuinely useful for explicit attacks, but it is not yet a strong semantic security boundary.

Slide 1

Opening and Headline

开场与项目总判断 · Suggested pace 建议时长：30–40 sec

Speaking goal 口播目标 让老师在第一分钟内知道：你测的是 Aegis，方式是真实网关测试，结论是“快，但不是强安全边界”。

建议话术 / Suggested speaking script

中文：感谢老师给我这个探索任务。这一周我主要评估的是 Aegis，一个面向 AI agent 工具调用的预执行防火墙。它的核心理念是，在 agent 真正执行 shell、文件、网络或数据库这类工具之前，先拦截这次调用并做安全判断。我这次不是只跑 demo，而是直接对真实 Aegis gateway 做了系统化测试。我想回答的问题很直接：在对抗性场景和真实 agent workflow 下，Aegis 到底能挡住多少，又会漏掉什么。先说结论：它对显式、单步、规则化攻击是有效的，而且速度很快；但对于语义攻击、伪装调用和多步链式攻击，还不能算一个足够强的安全边界。

English: Thank you for the opportunity. This week I evaluated Aegis, a pre-execution firewall for AI agent tool calls. Its core idea is to intercept a tool request before the agent actually executes shell, file, network, or database actions, and then decide whether that request is safe. I did not just run a demo. I tested the real Aegis gateway with a systematic evaluation setup. The question I wanted to answer was simple: under adversarial conditions and realistic agent workflows, what can Aegis reliably stop, and what still gets through? My top-line conclusion is that it works well for explicit, single-step, rule-shaped attacks and it is very fast, but it is not yet a strong security boundary against semantic attacks, disguised actions, or multi-step chains.

重点解释 / Key points to explain

- **Pre-execution firewall：**指的是“在工具真正执行之前”做决策，而不是等工具跑完后再报警。这一点很关键，因为它属于 runtime guardrail，而不是普通日志系统。
  - *English note:* This means security enforcement happens before execution, not after. That makes Aegis a runtime guardrail rather than a passive audit system.
- **Why this matters：**Agent 的风险不只是回答错，而是会不会真的去读文件、跑命令、联网或外传数据。
  - *English note:* For agents, the risk is not only bad text generation, but whether the model can trigger real file, shell, network, or exfiltration actions.
- **Headline phrasing：**“Fast but not a strong boundary” 这句话是整场汇报的主线，后面所有结果都要往这句话上收。
  - *English note:* The phrase “fast but not a strong boundary” should anchor the whole presentation and tie the later evidence together.

Slide 2

Professor Requirement Mapping

老师要求与交付对应关系 · Suggested pace 建议时长：35–45 sec

Speaking goal 口播目标 明确告诉老师：原始任务的每一项都已经落实，而不是只做了部分测试。

建议话术 / Suggested speaking script

中文：我把这一周的工作严格对齐到老师最初给的任务。第一，是 install and test the existing tool，所以我先把真实 Aegis 跑起来，再用可复现 harness 去压它，而不是自己写一个模拟规则。第二，是 report findings，所以我整理了 executive summary、technical summary、findings log、test matrix 和 results.json。第三，是 identify limitations and advanced attack vectors，所以我不仅复现了官方宣称的防护，还专门去做 prompt injection、外传、tool alias 和 multi-step abuse。第四，是 screenshots and simple demos，所以我保留了 dashboard、blocked case、bypass case 和 benign workflow 的证据。第五，是 test on OpenClaw or other agentic tools，所以我做了 DeepSeek function-calling agent 的直接接入路径，也用 OpenClaw 做了真实 agent framework 验证。

English: I structured the week strictly around the original task. First, “install and test the existing tool” meant I brought up the real Aegis gateway and stressed that live system with a reproducible harness, instead of testing a mock policy. Second, “report findings” meant I produced an executive summary, a technical summary, a findings log, a test matrix, and machine-readable results. Third, “identify limitations and advanced attack vectors” meant I went beyond baseline advertised protections and intentionally probed prompt injection, exfiltration, tool aliasing, and multi-step abuse. Fourth, “use screenshots and simple demos” meant I kept dashboard evidence, blocked cases, bypass cases, and benign workflow evidence. Fifth, “test on OpenClaw or other agentic tools” was satisfied through a DeepSeek function-calling agent as the direct Aegis path and OpenClaw as real framework validation.

重点解释 / Key points to explain

- **Requirement mapping：**这一页的功能是先消除老师对“你有没有偏题”的担心。
  - *English note:* This slide reassures the professor that the work directly answers the assigned task.
- **Artifact mindset：**你不是只做临时实验，而是把结果整理成可复用、可汇报、可扩展的 artifact。
  - *English note:* This shows the work is not just an ad-hoc experiment, but a reusable and presentable evaluation artifact.
- **Agentic validation：**这里要主动说明你不是只测静态 payload，也测了真实 agent loop。
  - *English note:* Emphasize that you validated both static cases and real agent loops, not just isolated payload strings.

Slide 3

System and Evaluation Setup

系统定义与测试设置 · Suggested pace 建议时长：45–55 sec

Speaking goal 口播目标 让老师清楚系统边界：Aegis 在哪里拦截、你的 harness 如何接入、主轨和对比轨分别是什么。

建议话术 / Suggested speaking script

中文：在系统定义上，我把 Aegis 视为一个 tool-call pre-execution firewall。执行路径可以简化成：用户提出请求，LLM 决定要不要调用工具，工具调用先经过 Aegis，Aegis 返回 allow、block 或 pending，然后工具才决定是否真正执行。我的主轨是本地真实 Aegis Docker gateway，也就是 localhost:8080。测试方式是统一的 pytest harness，直接 POST 到 /api/v1/check。除此之外，我还加了一条对比轨，用同样的 harness 去跑 DeepSeek-Chat baseline，这样就能比较规则引擎和语义判断之间的 trade-off。

English: At the system level, I treated Aegis as a tool-call pre-execution firewall. The execution path is: a user makes a request, the LLM decides on a tool call, that tool request is sent to Aegis first, Aegis returns allow, block, or pending, and only then does execution proceed. My primary track used the real Aegis Docker gateway running locally on localhost:8080. The harness was a unified pytest setup that posted directly to /api/v1/check. In addition, I added a comparison track with the same harness using a DeepSeek-Chat baseline, so I could compare rule-based enforcement against a more semantic checker.

重点解释 / Key points to explain

- **Primary vs comparison：**主结果永远以真实 Aegis gateway 为准；DeepSeek 只是对照组。
  - *English note:* The real Aegis gateway is the primary result. DeepSeek is only a comparison baseline.
- **Why unified harness matters：**同一套 harness 可以减少“测试方式不同导致结果不可比”的问题。
  - *English note:* A shared harness reduces comparability issues between the two tracks.
- **Runtime guardrail：**你测的不是模型回答质量，而是模型准备做工具调用时，安全层能否拦住它。
  - *English note:* The target is not answer quality but runtime enforcement at the moment a tool call is about to happen.

Slide 4

Coverage Across 7 Categories

七类测试覆盖面 · Suggested pace 建议时长：45–60 sec

Speaking goal 口播目标 说明这不是零散红队，而是有 taxonomy、有覆盖结构的系统测试。

建议话术 / Suggested speaking script

中文：测试覆盖一共分成七类。Baseline 用来复现 Aegis 宣称的基本能力；Prompt Injection 用来测最关键的 instruction hijacking 风险；Encoding 类看它是否只是拦明显字符串，而不是理解等价表达；File and Network 类关注路径、外联、SSRF、shell 注入这类系统层风险；Exfiltration 看敏感数据能不能被偷偷送出去；Multi-step 关注单次不危险、组合起来危险的链式攻击；Workflow 类则专门看正常开发流程下的误报和可用性。这样设计的目的，是把“能不能防住攻击”和“会不会把正常工作流也打坏”一起测清楚。

English: The evaluation covered seven categories. Baseline cases reproduced Aegis’ advertised protections. Prompt injection targeted the most important instruction-hijacking threat. Encoding cases asked whether Aegis only blocks obvious strings rather than semantically equivalent variants. File and network cases focused on system-level risks such as path abuse, outbound requests, SSRF, and shell injection. Exfiltration cases tested whether sensitive data could still be sent out. Multi-step cases examined chains where individual calls may look harmless but the whole sequence is dangerous. Workflow cases measured false positives and usability under normal development operations. The reason for this design is to measure both security and practical usability in the same artifact.

重点解释 / Key points to explain

- **Why categories matter：**分类不是为了好看，而是为了后续能分析“它到底在哪一类最脆弱”。
  - *English note:* The categories are essential because they allow you to localize where the system fails most often.
- **Workflow category：**安全系统如果只会全拦，其实没有实用价值，所以一定要测 benign workflow。
  - *English note:* A system that blocks everything is not useful, so benign workflow testing is part of the evaluation, not an afterthought.
- **Multi-step significance：**很多 agent 风险不是单个调用，而是一条链路；这一类是你项目的一个亮点。
  - *English note:* Many agent risks emerge from chains rather than single calls; that is one of the stronger parts of your project.

Slide 5

Headline Results and Data Convention

核心结果与数据口径 · Suggested pace 建议时长：55–70 sec

Speaking goal 口播目标 把最关键的数据一次讲清，同时避免老师对口径产生疑问。

建议话术 / Suggested speaking script

中文：主结果，也就是 Track A，来自真实 Aegis 规则引擎：84 个测试里，65 个被拦截，18 个确认绕过，3 个误报，平均延迟大约 5 毫秒。对比组，也就是 Track B，是 DeepSeek-Chat baseline：80 个测试里，73 个被拦截，5 个绕过，0 个误报，但平均延迟大约 2774 毫秒。这里最重要的不是单个数字，而是 trade-off：规则系统非常快，接近实时；但在语义攻击上更脆弱。LLM baseline 在语义检测上更强，但延迟高很多。关于 false positive，我这里统一采用最新仓库口径，也就是 3 个误报；原因是修复限流重试后，额外确认了几类正常 DevOps 操作也会被误报。

English: The main result, Track A, comes from the real Aegis rule engine: out of 84 tests, 65 were blocked, 18 were confirmed bypasses, and 3 were false positives, with an average latency of about 5 milliseconds. The comparison track, Track B, used the DeepSeek-Chat baseline: out of 80 tests, 73 were blocked, 5 bypassed, and there were 0 false positives, but average latency rose to about 2774 milliseconds. The key point is not any single number but the trade-off: the rule engine is extremely fast and close to real-time, but weaker against semantic attacks. The LLM baseline is much stronger semantically, but it is far more expensive in latency. For false positives, I use the latest repository convention of 3, because after fixing rate-limit retry handling, a few additional normal DevOps actions were confirmed as false positives.

重点解释 / Key points to explain

- **Data track wording：**要主动说“Track A 是主结果，Track B 仅作对比”，防止老师把两套数字混起来。
  - *English note:* State clearly that Track A is the primary result and Track B is comparison only.
- **Latency trade-off：**大概 5ms 对 2774ms，是“工程可用性”和“语义理解能力”的典型权衡。
  - *English note:* Roughly 5ms versus 2774ms is a classic trade-off between deployable speed and semantic understanding.
- **False positives：**误报不是小问题，因为它会影响正常工作流，也说明规则写得过于粗糙。
  - *English note:* False positives are important because they harm usability and expose overly coarse rule logic.

如果老师追问限流问题 / If asked about rate limiting

中文：最初的 harness 在高频请求下把 rate-limit response 错当成 allow。后来我改成了自动重试，所以最终统计口径更可靠。

English: The original harness treated rate-limit responses as if they were allows under rapid requests. I later fixed that with auto-retry, so the final counts are more reliable.

Slide 6

Category Breakdown

分类型结果解读 · Suggested pace 建议时长：45–55 sec

Speaking goal 口播目标 把“哪里最容易失守”讲成有层次的分析，而不是简单念表格。

建议话术 / Suggested speaking script

中文：从分类型结果来看，最突出的弱点是 Prompt Injection：13 个测试里有 7 个绕过。这不是边缘问题，而是 agent security 的核心 threat model。其次是 Exfiltration，7 个里有 4 个绕过，说明它对“数据是否敏感、是否在分步外传”这件事缺乏足够理解。File and Network 里也有 4 个绕过，多数都和 policy routing 或上下文缺失有关。相比之下，Baseline 和 Encoding 相对更强，这说明 Aegis 对显式、模板化、已知模式的攻击还是有不错覆盖的。也就是说，问题不在于它完全没用，而在于它的保护边界比较窄。

English: Looking at the category breakdown, the most serious weakness is prompt injection: 7 bypasses out of 13 tests. This is not a side issue; it is one of the core threat models in agent security. Exfiltration is the next major weakness, with 4 bypasses out of 7, which suggests the system does not adequately understand data sensitivity or multi-step leakage. File and network testing also produced 4 bypasses, mostly tied to policy routing or lack of context. By contrast, baseline and encoding performance is much stronger, which shows that Aegis does cover explicit, template-like, and already-known attack patterns reasonably well. So the problem is not that Aegis is useless; the problem is that its protection boundary is relatively narrow.

重点解释 / Key points to explain

- **Prompt injection as headline weakness：**这一页必须把 PI 定位成“最大单点弱项”，因为这会自然引到后面的 root cause。
  - *English note:* Prompt injection should be framed as the single largest weakness because it leads naturally into the root-cause analysis.
- **Exfiltration meaning：**外传绕过高，说明系统不只缺 regex，还缺“数据流”和“目标语义”的判断。
  - *English note:* High exfiltration bypasses indicate a need not just for regexes, but for data-flow and destination-aware reasoning.
- **Boundary mapping：**你的表述要成熟：不是说它不行，而是说它在哪些威胁模型下不够。
  - *English note:* Frame this as capability-boundary mapping rather than blanket criticism.

Slide 7

Evidence: System Running and Explicit Blocking

证据：系统在线且显式攻击能被拦住 · Suggested pace 建议时长：35–45 sec

Speaking goal 口播目标 告诉老师你的证据链是完整的：系统真的跑起来了，而且不是所有攻击都漏掉。

建议话术 / Suggested speaking script

中文：这一页主要承担两个证明作用。第一，dashboard 和健康状态说明真实 Aegis 环境是在线的，不是纸面分析。第二，被拦住的经典 prompt injection 说明 Aegis 不是完全失效，它对字面上明显的攻击确实有效。这个点很重要，因为如果一个系统什么都拦不住，那后面就没有分析价值了；但如果它能拦住一部分、又漏掉更隐蔽的攻击，我们就能更清楚地画出它的边界。

English: This slide provides two forms of evidence. First, the dashboard and health view show that the real Aegis environment was actually running; this is not a paper-only analysis. Second, the blocked classic prompt injection case shows that Aegis is not completely ineffective. It does work on obvious, literal attacks. That matters because if a system blocks nothing, there is not much to analyze. But if it blocks some attacks while missing more advanced ones, then we can draw a much more meaningful boundary of what it does and does not protect against.

重点解释 / Key points to explain

- **Evidence chain：**你是在讲“真实环境 + 实际决策结果 + 可视化证据”，这是可信度来源。
  - *English note:* Your credibility comes from the chain of real environment, real decisions, and visual evidence.
- **Balanced framing：**要避免把 Aegis 说成完全没用；这一页就是平衡叙事。
  - *English note:* Use this slide to avoid an overly negative framing and keep the story balanced.
- **Literal attacks：**“字面攻击能拦”是后面“语义攻击会漏”的对照组。
  - *English note:* Literal blocking serves as the control case for the later claim that semantic attacks still slip through.

Slide 8

Evidence: Bypass and Benign Workflow

证据：真实绕过与正常工作流并列展示 · Suggested pace 建议时长：40–50 sec

Speaking goal 口播目标 让老师同时看到安全性和可用性，而不是只盯着攻击。

建议话术 / Suggested speaking script

中文：这一页把两个方向放在一起看。左边的绕过案例是 paraphrased prompt injection，也就是老师特别希望我去测的“更高级一点”的攻击：不再写经典短语，而是换一种语义表达方式，结果真实网关允许通过。右边的 benign workflow 则说明我也测了正常场景，不是只做红队。因为一个真正有价值的防护层，必须在安全性和可用性之间找到平衡。如果它把所有正常流程都打断，那就不是好的 agent safeguard。

English: This slide places two perspectives side by side. On the left is a real bypass: a paraphrased prompt injection case, which is exactly the more advanced attack the professor wanted me to probe. Instead of using the classic trigger phrase, the attack was reworded semantically, and the real gateway allowed it. On the right is a benign workflow, which shows that I also evaluated normal behavior rather than only red-team breakage. A useful defense layer must balance security and usability. If it breaks every normal workflow, it is not a good safeguard for agents.

重点解释 / Key points to explain

- **Paraphrase-level attack：**这里要强调：不是简单换标点，而是语义上等价、表面上不同。
  - *English note:* Emphasize that the bypass is not a trivial punctuation change but a semantically equivalent rewording.
- **Why benign matters：**汇报中主动提 usability，会让老师觉得你的判断更成熟。
  - *English note:* Calling out usability explicitly makes your evaluation sound more mature and practical.
- **Balanced story：**你在说的是“能用，但不够稳健”，不是“完全失败”。
  - *English note:* The message is “usable but not robust enough,” not “total failure.”

Slide 9

Four Representative Advanced Bypasses

四类最能说明问题的高级绕过 · Suggested pace 建议时长：65–80 sec

Speaking goal 口播目标 用四个案例把主要 failure modes 讲全，并且每个都能自然引到后续研究方向。

建议话术 / Suggested speaking script

中文：在所有绕过里，我挑了四个最有代表性的。第一类是 prompt injection paraphrase，说明系统太依赖字面短语。第二类是 tool alias confusion，比如 maintenance_helper 这种名字听起来很正常，但参数语义其实很危险，结果系统走错了 policy family。第三类是 trusted-domain exfiltration，也就是 URL 看起来合法，比如发到 GitHub Gist 或 Slack webhook，但真正危险的是 body 里的敏感内容。第四类是 chunked exfiltration，每一次请求都很小、看上去也不明显，但整条链组合起来就是外传攻击。这四类案例之所以重要，是因为它们分别对应语义不足、路由不足、上下文不足和跨调用状态不足，刚好能概括 Aegis 目前的设计边界。

English: Among all bypasses, I selected four representative ones. The first is prompt injection paraphrase, which shows that the system relies too heavily on literal trigger phrases. The second is tool alias confusion: a name like maintenance_helper sounds harmless, but its arguments can still imply dangerous behavior, and the system ends up in the wrong policy family. The third is trusted-domain exfiltration, where the destination URL looks legitimate, such as GitHub Gist or a Slack webhook, but the real danger is the sensitive content in the body. The fourth is chunked exfiltration, where each request looks small and acceptable, but the full sequence becomes an exfiltration attack. These four matter because together they map to semantic weakness, routing weakness, context weakness, and lack of cross-call state—essentially the current design boundary of Aegis.

重点解释 / Key points to explain

- **Representative cases：**这里不是在堆漏洞，而是在挑最能代表设计缺口的四类。
  - *English note:* You are not listing random bugs; you are selecting four cases that represent broader design gaps.
- **Trusted-domain abuse：**这很值得强调，因为它说明光看 URL 或 domain 不够，body semantics 也重要。
  - *English note:* This matters because URL-level legitimacy is not enough; the content semantics also matter.
- **Chunked exfiltration：**这一类最能体现为什么需要跨调用状态和 taint tracking。
  - *English note:* This is the clearest motivation for session-level state and taint tracking.

讲 demo intuition 时怎么说 / How to explain the demo intuition

中文：可以补一句：同样的危险意图，换一个工具名字或包装到另一个字段里，规则就可能不触发。

English: You can add that the same dangerous intent may be missed once it is wrapped in a different field or under a different tool name.

Slide 10

Root Causes: Three Design-Level Gaps

根因分析：三个设计层缺口 · Suggested pace 建议时长：60–75 sec

Speaking goal 口播目标 把“发现了什么”上升成“为什么会这样”，这是研究味最强的一页。

建议话术 / Suggested speaking script

中文：我把这些绕过总结成三个设计层根因。第一，prompt-injection policy 太 literal，主要依赖少量精确短语，而且只看有限字段，所以一旦换字段名、做改写或间接注入，就容易漏。第二，tool name 驱动 policy routing，也就是说系统先按工具名字猜这是什么风险，再套对应规则；如果名字和真实行为不一致，就会走错路。第三，per-call stateless evaluation，也就是一次只看一个调用，看不到整条工作流，所以 read-then-exfil、chunked leakage、multi-turn setup 这类链式行为很难识别。我的核心判断是：问题不是少写了一个 regex，而是现有 rule engine 缺少语义理解和跨调用上下文。

English: I summarized the bypasses into three design-level root causes. First, the prompt-injection policy is too literal: it relies on a small set of exact trigger phrases and limited field coverage, so once the attack changes field names, rephrases itself, or arrives indirectly, the rule often does not fire. Second, policy routing is heavily driven by tool names. In other words, the system first guesses the risk family from the tool label and then applies that policy. If the name does not match the actual behavior, the system follows the wrong path. Third, evaluation is stateless on a per-call basis. The engine sees one call at a time, not the whole workflow, so chains such as read-then-exfil, chunked leakage, or multi-turn setup are hard to detect. My main conclusion is that the issue is not a missing regex. The current rule engine lacks semantic understanding and cross-call context.

重点解释 / Key points to explain

- **Literal matching：**把“字面匹配”说清楚，老师就会明白为什么 paraphrase 会绕过。
  - *English note:* Once you explain literal matching, the prompt-injection bypass becomes intuitively clear.
- **Policy routing：**这不是简单误报，而是更深层的 architecture choice。
  - *English note:* This is not just a false positive issue; it is an architectural choice with security consequences.
- **Cross-call context：**这一点最接近真正的 research question：agent security 需要看单步还是看链路？
  - *English note:* This points directly to a larger research question: should agent security reason per step or across full chains?

Slide 11

Research Value and Next Steps

研究价值与后续方向 · Suggested pace 建议时长：45–60 sec

Speaking goal 口播目标 把项目从“安全测试”提升成“后续能继续做的研究方向”。

建议话术 / Suggested speaking script

中文：基于这些结果，我觉得后续最自然的方向有三个。第一，是加一个 lightweight semantic layer，不一定每次都上大模型，而是在模糊的 prompt injection 或外传场景下，给规则系统补一层更强的语义判断。第二，是做 taint 或 provenance tracking，对从 .env、.ssh 或云凭证里读出来的数据打标签，然后跟踪它后续是否被外传。第三，是把这套 harness 和 scoring 继续整理成 benchmark，用来比较不同 agent runtime guard system。这样一来，这个项目就不只是一次性试工具，而是能发展成工程和研究都成立的方向。

English: Based on the findings, I see three natural next steps. First, add a lightweight semantic layer. That does not mean using a large model for every call, but rather applying stronger semantic checking only to ambiguous prompt injection or exfiltration cases. Second, introduce taint or provenance tracking so that data read from sources like .env, .ssh, or cloud credentials can be labeled and monitored for later leakage. Third, turn the harness and scoring into a benchmark for comparing different runtime guard systems for agents. That would move the project beyond a one-off tool test and into something meaningful for both engineering and research.

重点解释 / Key points to explain

- **Lightweight semantic layer：**这句话体现你的想法不是“全都交给 LLM”，而是做 hybrid defense。
  - *English note:* This shows that your idea is not “just use an LLM everywhere,” but rather a hybrid defense architecture.
- **Taint tracking：**这是把安全问题从“字符串匹配”提升到“数据流控制”。
  - *English note:* Taint tracking reframes the problem from string matching to data-flow control.
- **Benchmark angle：**老师如果想继续带你做，这个方向最容易扩展成长期工作。
  - *English note:* The benchmark angle is especially strong if the professor is thinking about longer-term RA work.

Slide 12

Agentic Tool Validation

真实 agent 工具验证 · Suggested pace 建议时长：55–70 sec

Speaking goal 口播目标 说明你不是只停留在静态 payload，而是看了真实 agent loop 里的防御链。

建议话术 / Suggested speaking script

中文：除了 pytest 之外，我还做了真实 agentic validation。第一条线是 DeepSeek function-calling agent，我把它的 tool decisions 通过 Aegis 的 /api/v1/check 做预检查，再决定是否执行。这里可以看到 benign task 会被允许，base64 编码攻击和 SSRF 会被拦住，而一些更明显的危险请求甚至会先被 LLM 自己拒绝。第二条线是 OpenClaw CLI。它让我验证，在真实 agent framework 里，安全其实是多层的：LLM 自拒绝是一层，Aegis 规则阻断是一层，框架自身的 URL filter 或 native guard 也是一层。没有任何单层足够，所以我最后把这一页总结成 defense-in-depth。

English: Beyond pytest, I also performed real agentic validation. The first path used a DeepSeek function-calling agent, where tool decisions were routed through Aegis /api/v1/check before simulated execution. In that setup, benign tasks were allowed, while base64-encoded attacks and SSRF were blocked, and some obviously dangerous requests were even self-refused by the LLM before the tool layer. The second path used OpenClaw CLI. That let me validate that in a real agent framework, security is layered: LLM self-refusal is one layer, Aegis rule-based blocking is another, and the framework’s own URL filters or native safeguards are yet another. No single layer is sufficient, which is why I summarize this slide as defense in depth.

重点解释 / Key points to explain

- **Direct vs indirect validation：**DeepSeek function-calling 是直接经过 Aegis；OpenClaw 更像框架级验证。这个区分一定要讲清楚。
  - *English note:* DeepSeek function-calling is the direct Aegis path, while OpenClaw is primarily framework-level validation. Keep that distinction clear.
- **Defense in depth：**这一页最好的收束方式就是“多层防御，而不是单点神话”。
  - *English note:* The best closing frame here is defense in depth rather than a single magic security layer.
- **Why this matters：**老师会在意实际可落地性，这一页能说明你看到了真实系统里的复杂性。
  - *English note:* This slide matters because it shows you understand deployment reality, not just isolated test cases.

Slide 13

Final Takeaway

总结与收尾 · Suggested pace 建议时长：30–40 sec

Speaking goal 口播目标 用一句成熟的话把整场报告收住，并自然留出后续合作空间。

建议话术 / Suggested speaking script

中文：最后总结一下：这一周我完成了真实 Aegis gateway 的安装、系统化测试、结果整理和 agentic validation，也找到了几类比较有代表性的高级绕过。我的判断不是 Aegis 没有价值，而是它目前更适合做 fast runtime filter，去拦显式、单步、规则化攻击；如果想进一步面对语义攻击、伪装调用和多步外传，还需要更强的 semantic layer、cross-call context 和 hybrid defense 设计。也就是说，这个方向本身是有价值的，而且我觉得很适合继续往 attacking and defending agentic systems 这个方向深入。

English: To conclude, this week I installed and tested the real Aegis gateway, built a systematic evaluation artifact, organized the results, and validated the findings with agentic tools. I also identified several representative advanced bypasses. My conclusion is not that Aegis lacks value. Rather, it is currently better understood as a fast runtime filter for explicit, single-step, rule-shaped attacks. If the goal is to defend against semantic attacks, disguised actions, and multi-step exfiltration, then stronger semantic layers, cross-call context, and hybrid defense design are still needed. In that sense, the direction itself is promising, and I think it naturally opens up further work on both attacking and defending agentic systems.

重点解释 / Key points to explain

- **Do not overstate：**最后不要说“我把它攻破了”，要说“我测清了它的边界”。
  - *English note:* Do not say “I broke it.” Say that you mapped its current boundary.
- **Future-oriented close：**收尾要给老师一个信号：这件事可以继续做，不是一次性展示。
  - *English note:* End in a way that signals this can grow into sustained research or RA work.
- **Memorable sentence：**最推荐记住的一句是：Aegis is fast, useful, and operationally real—but not yet a strong semantic security boundary.
  - *English note:* A good closing sentence to memorize is: “Aegis is fast, useful, and operationally real—but not yet a strong semantic security boundary.”

Slide Appendix

Quick Recall Sheet

速记页 · Suggested pace 建议时长：for rehearsal

Core numbers 核心数字 Track A: 84 tests / 65 blocked / 18 bypass / 3 false positives / ~5 ms. Track B: 80 tests / 73 blocked / 5 bypass / 0 false positives / ~2774 ms.

最值得背熟的四句话 / Four lines worth memorizing

- **Line 1：**Aegis 是一个 tool-call pre-execution firewall，核心价值是 runtime guardrail。
  - *English note:* Aegis is a tool-call pre-execution firewall, and its core value is runtime enforcement.
- **Line 2：**它对显式、单步、规则化攻击有效，但对语义攻击和链式攻击不够稳。
  - *English note:* It is effective for explicit, single-step, rule-shaped attacks, but less robust against semantic and chained attacks.
- **Line 3：**最大弱点是 prompt injection，其次是 exfiltration、tool alias confusion 和缺少 cross-call context。
  - *English note:* The main weakness is prompt injection, followed by exfiltration, tool alias confusion, and lack of cross-call context.
- **Line 4：**最自然的后续方向是 hybrid defense：轻量语义层 + taint tracking + benchmark 化。
  - *English note:* The most natural next step is a hybrid defense: lightweight semantics, taint tracking, and benchmarkization.